Exact balanced minimum evolution evaluation and parallel search in Phylo2Vec coordinates

Abstract

Balanced minimum evolution (BME) provides a distance-based criterion for phylogenetic inference, but searching large tree spaces requires repeated evaluation of many candidate rearrangements. We formulate exact BME evaluation as discrete coordinate differences in Phylo2Vec space and organize the underlying subtree-distance recurrences for parallel execution. For a tree represented in an ordered labeling, all valid single-coordinate changes are evaluated in $O(N^2)$ time. Relabeling preserves the current tree while changing the coordinate neighborhood. The same evaluator applies to smaller skeleton trees whose leaves represent fixed components of the full tree. We prove that exact balanced distances between these components preserve every skeleton-induced BME change. The reduced objective differs from the full objective only by a constant, so each minibatch coordinate difference is the corresponding full-tree difference. This reduction also covers a subtree window when its entire complement is retained as an oriented component, with outside distances updated after each committed window. Independent small-tree checks agree with direct BME differences to maximum absolute discrepancies of 3.553×10^{-14} for coordinate evaluation and 1.564×10^{-13} for condensation. In 120 small-tree comparisons, the CPU optimizer matches or improves the reference BME score in 119 cases. A full gradient sweep at $N = 40{,}000$ takes 158.36 ms on eight H100 GPUs. Exactness is an algebraic property relative to the specified distance matrix. Numerical evaluation remains subject to floating-point rounding, and the search remains heuristic.

1 Introduction

Distance-based phylogenetic inference separates the estimation of pairwise evolutionary distances from the search for a tree. Balanced minimum evolution (BME) scores a topology using a weighted sum of those distances, with weights determined by the branching structure along each leaf-to-leaf path [1, 2, 3]. Efficient recurrences permit an entire subtree-prune-and-regraft (SPR) neighborhood to be evaluated in quadratic time [4, 5]. For large trees, the practical costs include storing the subtree-distance table, repeatedly traversing the tree, and coordinating many small candidate evaluations. The number of optimization sweeps depends on the instance and the search strategy.

Phylo2Vec represents rooted binary trees by bounded integer vectors [6]. This representation provides a coordinate system in which a discrete difference records the change in BME after replacing one vector entry. An ordered labeling permits efficient decoding and maps coordinate changes to a structured collection of SPR moves. The representation also separates the tree topology from its leaf-label permutation. Changing the permutation can expose different coordinate moves without changing the current labeled tree or its score.

Our method combines this coordinate formulation with parallel evaluation and component-based reduction. The reduction relies on established BME structure. Balanced subtree averages underlie earlier distance-based algorithms [2, 3], and fixed-clade families have been described as affine images of smaller BME polytopes [7, Section 2.4]. We state the corresponding objective identity explicitly because its assumptions determine when minibatch optimization evaluates the full-tree objective. In particular, condensed distances must include the balanced contributions of every leaf, and a subtree window must retain the rest of the tree through the correct attachment orientation.

The contribution developed here is an algorithmic formulation: exact BME coordinate evaluation, its extension to minibatches through exact condensed distances, and parallel execution using column decomposition. We distinguish exact identities from finite-precision calculations and from heuristic search decisions. The main experiments assess objective agreement, local optimization, and gradient throughput. Recorded sequence-refinement measurements of a separate implementation variant are identified in Appendix C.

2 Methods

2.1 BME and Phylo2Vec coordinates

Let T be an unrooted binary tree on $N \geq 3$ labeled leaves, and let D be a fixed, finite, symmetric matrix with zero diagonal. Write $q_T(x, y)$ for the number of internal degree-three vertices on the path from x to y. The BME objective is

$$L(T; D) = \sum_{0 \leq x < y < N} D_{xy}\, 2^{-q_T(x,y)}. \tag{1}$$

If the path has $e_T(x, y)$ edges, then $q_T(x, y) = e_T(x, y) - 1$. Thus the unordered-pair coefficient is $2^{1-e_T(x,y)}$. An artificial degree-two root used to encode the tree is suppressed before evaluating these path lengths. No additivity assumption on D is needed for the identities below.

The Phylo2Vec domain for rooted binary trees is

$$\mathcal{V}_N = \{(v_0, \ldots, v_{N-2}) : 0 \leq v_i \leq 2i\}, \qquad |\mathcal{V}_N| = (2N - 3)!!. \tag{2}$$

We use the ordered, or cherry-first, subset $\mathcal{V}_N^{\mathrm{cf}} = \{v : 0 \leq v_i \leq i\}$ as the baseline for coordinate evaluation. Every rooted binary tree can be represented in this subset after a suitable leaf permutation. The relabeling construction and its proof are given in Appendix A. If $\pi(a)$ is the original leaf represented by current index a, the distance matrix in these coordinates is $D_\pi[a, b] = D[\pi(a), \pi(b)]$. Permuting both axes in this way preserves the objective. Constructing the permutation takes $O(N)$ time from a decoded tree, while explicitly copying a dense permuted matrix takes $O(N^2)$ time.

For an ordered baseline v, define the discrete coordinate difference

$$\Delta_{i,j} = L(T(v^{(i \leftarrow j)}); D_\pi) - L(T(v); D_\pi), \qquad 0 \leq j \leq 2i. \tag{3}$$

There are $(N - 1)^2$ valid coordinate entries, including unchanged coordinates and entries that can represent the same unrooted topology. This count is not a count of distinct SPR neighbors. General candidate values $j > i$ are allowed even though the baseline is ordered. After accepting such a move, the resulting tree is decoded and relabeled before the next ordered evaluation.

2.2 Quadratic-time coordinate evaluation

For two distinct vertices u, w, define $A(u, w)$ as the balanced average distance between the two components oriented away from the path joining u and w. At an internal vertex, the two outward branches each receive weight $1/2$. At a leaf, the corresponding distribution is concentrated on that leaf. Consequently, when the component at u has children a, b,

$$A(u, w) = \tfrac{1}{2}\{A(a, w) + A(b, w)\}. \tag{4}$$

The direction is important when one vertex is an ancestor of the other in a computational rooting. In that case, the component away from w combines the sibling branch and the parent branch:

$$A(u, w) = \tfrac{1}{2}\{A(\mathrm{sib}_u(w), w) + A(\mathrm{parent}(u), w)\}. \tag{5}$$

At the degree-three computational root, the analogous average uses the two branches not containing w. Bottom-up and top-down passes construct the required table in $O(N^2)$ arithmetic operations.

For a fixed pruned component, the standard BME SPR recurrence propagates score differences across regraft edges in $O(N)$ time [4]. Each propagation step is an adjacent interchange of four directed components. Its score change is a linear combination of their balanced distances, with coefficient $1/4$. Successive steps accumulate the change relative to the original attachment. Phylo2Vec supplies the mapping from each valid coordinate value to one of these regraft edges. The explicit mapping, including the collapsed-root boundary, is in Appendix A.3.

Proposition 2.1 (Cost and scope of a coordinate sweep)**.** *Given an ordered baseline and its leaf permutation, all entries in Equation (3) can be computed in $O(N^2)$ time. In exact arithmetic, these entries equal the corresponding full-tree BME differences.*

Proof. The balanced-distance table costs $O(N^2)$. For each of $O(N)$ coordinates, pruning its associated component and traversing the two remaining half-trees visits $O(N)$ edges. The SPR recurrence evaluates the score at each edge, and the coordinate mapping selects the appropriate values without evaluating the candidate trees independently. The mapping costs at most $O(N)$ per row. The recurrence preserves the Pauplin score difference at each traversal step, so selecting a mapped edge gives Equation (3). □

The proposition concerns the coordinate neighborhood of the current labeling. It does not assert that this neighborhood is the entire SPR neighborhood, or that coordinate descent finds a global BME optimum.

2.3 Exact condensed distances and skeleton reduction

Partition all N leaves into $K \geq 3$ fixed components $S_1, \ldots, S_K$. Each component has a fixed internal binary topology and a single attachment to the rest of the tree. A component can consist of one leaf. For a non-singleton component, let r_a be the internal vertex adjacent to its attachment edge, and let $h_a(x)$ count the branching vertices from r_a to leaf x, including r_a. For a singleton, set $h_a(x) = 0$. Define its balanced leaf weights by

$$\omega_a(x) = 2^{-h_a(x)}, \qquad x \in S_a. \tag{6}$$

Lemma 2.2 (Normalization of leaf weights)**.** *For each component, $\sum_{x \in S_a} \omega_a(x) = 1$.*

Proof. A singleton has weight one. At a binary root, every leaf weight in each child component is multiplied by $1/2$. Induction over the component therefore gives total mass $\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 = 1$. □

Let G be an unrooted binary skeleton with one leaf for each component. Construct T_G by replacing skeleton leaf a with S_a, retaining the attachment edge. Define a condensed distance matrix by

$$\widetilde{D}_{ab} = \sum_{x \in S_a} \sum_{y \in S_b} \omega_a(x)\omega_b(y)D_{xy} \quad (a \neq b), \qquad \widetilde{D}_{aa} = 0. \tag{7}$$

These are exact balanced averages of the original pairwise distances. They can be formed by the weighted sum above or, equivalently, by recursively averaging over the two children of each component root. All leaf contributions are retained. This is an objective-level form of the fixed-clade reduction discussed by Haws et al. [7, Section 2.4]. We include the proof to specify the weights and the scope of exactness needed by the algorithm.

Theorem 2.3 (BME decomposition over fixed components)**.** *There is a constant C, independent of the skeleton topology, such that*

$$L(T_G; D) = C + L(G; \widetilde{D}). \tag{8}$$

Consequently, for any two skeletons G and G',

$$L(T_{G'}; D) - L(T_G; D) = L(G'; \widetilde{D}) - L(G; \widetilde{D}). \tag{9}$$

Proof. Separate the unordered leaf pairs into pairs within a component and pairs between components. For $x, y \in S_a$, the path stays inside S_a and is unchanged by rearranging the skeleton. The sum of all such terms is

$$C = \sum_a \sum_{\substack{x<y \\ x,y \in S_a}} D_{xy}\, 2^{-q_{T_G}(x,y)},$$

which is independent of G.

For $x \in S_a$ and $y \in S_b$ with $a \neq b$, the path contains the branching vertices inside S_a, those on the skeleton path, and those inside S_b. These sets of vertices are disjoint, giving

$$q_{T_G}(x, y) = h_a(x) + q_G(a, b) + h_b(y).$$

Therefore its weight factors as $2^{-q_{T_G}(x,y)} = \omega_a(x)\, 2^{-q_G(a,b)}\, \omega_b(y)$. Summing between-component terms gives

$$\begin{aligned} \sum_{a<b} \sum_{\substack{x \in S_a \\ y \in S_b}} D_{xy}\, 2^{-q_{T_G}(x,y)} &= \sum_{a<b} 2^{-q_G(a,b)} \sum_{\substack{x \in S_a \\ y \in S_b}} \omega_a(x)\omega_b(y) D_{xy} \\ &= L(G; \widetilde{D}). \end{aligned}$$

Adding the within-component constant proves Equation (8). Subtracting two such decompositions cancels C and proves Equation (9). □

Thus a skeleton evaluates exact changes in the full objective, even though its absolute score omits C. The theorem applies to every rearrangement preserving the chosen components, not only to SPR moves. It also applies to arbitrary fixed dissimilarities, including non-additive estimated distances.

Corollary 2.4 (Exact minibatch coordinate differences)**.** *Let v encode an ordered skeleton G, keep the attached components fixed, and index $\widetilde{D}$ by the same leaf labeling. Evaluating a coordinate replacement $v_i \leftarrow j$ using the exact matrix $\widetilde{D}$ gives*

$$\Delta^{\text{skel}}_{i,j} = L(T_{G(v^{(i \leftarrow j)})}; D) - L(T_{G(v)}; D). \tag{10}$$

Once $\widetilde{D}$ is available, all valid skeleton coordinate differences can be evaluated in $O(K^2)$ time.

Proof. Apply Proposition 2.1 to the K-leaf skeleton with input $\widetilde{D}$. Equation (9) identifies each resulting skeleton difference with the corresponding full-tree difference. □

Corollary 2.5 (A proper subtree window)**.** *For a window attached to the rest of a binary tree by one edge, treat its entire complement as an additional component oriented away from that edge. If its internal topology and exact balanced distances to the other components are retained, Equation (9) applies to rearrangements of the window skeleton.*

Proof. Cutting the attachment edge produces two components. The endpoint in the complement has two outward branches unless the complement is a singleton. It therefore defines precisely a binary component of the type used in the theorem. Including it completes the partition of all leaves, so both within-window and window-to-complement contributions are included. □

The weights entering the outside distances are determined by the current topology of the complement and its attachment edge. They must be updated when a preceding rearrangement changes that topology.

2.4 Exact minibatch evaluation and sequential updates

With the fixed components and exact condensed matrix in place, candidate skeletons are evaluated against the full-tree objective through Corollary 2.4. The same matrix can be reused throughout a skeleton search because its component topologies and attachment roots remain fixed. For a candidate G', let $\mathcal{P}(G, G') = \{(a, b) : a < b,\ q_G(a, b) \neq q_{G'}(a, b)\}$. Its full-tree BME change can also be computed directly from the changed pairs:

$$\delta = \sum_{(a,b)\in\mathcal{P}(G,G')} \left(2^{-q_{G'}(a,b)} - 2^{-q_G(a,b)}\right) \widetilde{D}_{ab}. \tag{11}$$

Unchanged pairs contribute zero, and the within-component constant cancels. Theorem 2.3 therefore identifies δ with the full-tree BME change. Thus both coordinate selection and the evaluation of a combined candidate can use the same exact condensed distances.

Windows are committed sequentially. After a commit, affected ancestor summaries are updated and the next window's outside distances are constructed from the current tree. Node-disjoint windows need not have additive BME changes because both updates can alter the coefficients of distances between their leaves. For consecutive accepted trees $T^{(0)}, \ldots, T^{(M)}$, exact arithmetic gives

$$L(T^{(M)}; D) - L(T^{(0)}; D) = \sum_{m=1}^{M} \left[L(T^{(m)}; D) - L(T^{(m-1)}; D)\right]. \tag{12}$$

If each increment is negative, so is the cumulative increment. This argument requires that each acceptance calculation uses the current tree, rather than a shared pre-update tree.

The mathematical equalities are exact in real arithmetic. Computed distances, leaf weights, and reductions remain subject to rounding and, on sufficiently deep trees, underflow. Exactness here is relative to the specified distance matrix, not to unknown evolutionary distances.

2.5 Search strategy

The full-tree CPU optimizer evaluates coordinate differences, ranks improving rows, and forms candidate batches using disjoint path corridors as a conflict filter. It then evaluates the combined candidate tree and backtracks to a smaller batch if the score does not improve. The acceptance decision uses the joint objective rather than the sum of individual predicted improvements. When the current coordinate neighborhood is exhausted, the optimizer changes the ordered labeling and searches again. Its full-tree configuration also scans remaining unrooted SPR moves before declaring SPR-local convergence. An optional restart stage applies random perturbations and resumes descent while retaining the best tree found. Only accepted descent updates and the retained best score are monotone. The perturbed working trajectory need not be.

An exact minibatch search can alternate skeletons at different scales. A global skeleton rearranges the backbone, local windows expose detailed structure, and overlapping or paired windows propose changes between regions. Each window follows four steps:

1. Select fixed components from the current tree, including an oriented outside component for a proper subtree.

2. Form the exact condensed distance matrix in Equation (7).

3. Evaluate skeleton coordinate changes using Corollary 2.4, and check the selected update using Equation (11).

4. Commit an improving candidate, graft the components back, and update summaries before selecting or evaluating the next window.

These steps preserve the objective being optimized under the assumptions above. They do not prove that a particular finite window schedule explores every tree or reaches every improving move. Nor does an infinite number of batches alone establish such a result without assumptions on which proposals can occur and be accepted.

2.6 Parallel evaluation and computational cost

Equation (4) has an independent recurrence for each fixed column. Assigning columns to threads lets neighboring threads access consecutive entries while following the same traversal order. The top-down correction for column w follows its ancestor path. Depth-first entry intervals determine the next child without a traversal stack. Its total work over columns is $O(\sum_w \mathrm{depth}(w))$, bounded by $O(N^2)$ and reduced to $O(N \log N)$ for trees of logarithmic height.

A full balanced-distance table stores approximately $4N^2$ entries. The column-tiled variant retains the internal-to-leaf block and reconstructs required internal rows on demand. This reduces that table to approximately N^2 entries, while the input matrix and other buffers still contribute to total memory. A row-wise reduction can return the best coordinate per row without storing the rectangular gradient output. Columns and SPR rows can also be distributed across GPUs. Appendix B gives the dependency and workspace arguments.

For a skeleton with K leaves, a coordinate sweep costs $O(K^2)$ once its distances are available. This does not make arbitrary distance construction independent of N. Direct evaluation of one condensed pair can require $|S_a||S_b|$ original distances. Explicit leaf weights use $O(N)$ entries, and constructing exact condensed distances can dominate runtime for large component pairs. When $D_{xy} = \phi(x)^\top \psi(y)$ has a fixed-dimensional bilinear representation, weighted component embeddings give $\widetilde{D}_{ab} = (\sum_x \omega_a(x)\phi(x))^\top (\sum_y \omega_b(y)\psi(y))$. That shortcut is exact for the specified bilinear distance. It does not justify averaging embeddings before applying a nonlinear sequence-distance correction.

3 Experimental design

The main evaluation distinguishes three workloads. First, small-tree tests compare coordinate differences and condensed differences against independent evaluations of Equation (1). Second, CPU optimization experiments compare final scores and wall times from identical starting trees. Third, gradient benchmarks measure parallel sweep throughput, independently of convergence. The large embedding-based scaling workload is distinct from sequence-distance refinement. Appendix C separately reports sequence benchmarks of an implementation using approximate proposals and exact full-tree acceptance calculations. Those timings do not measure a minibatch search using exact condensed distances throughout.

The CPU comparison uses additive distances from trees whose branch lengths are uniform on $(0.05, 0.55)$ and a noisy variant $D'_{xy} = D_{xy} \exp(0.15 Z_{xy})$, with symmetric standard-normal Z_{xy}. There are respectively 20, 20, 10, 5, and 5 replicates at $N = 20, 50, 100, 150, 200$ in each regime. The reference is the project's in-memory C implementation of the FastME BME-SPR algorithm. Agreement with the external FastME executable is assessed separately. A score tolerance of 10^{-9} defines wins, ties, and losses. Single-core timings are reported for Apple M5 and AMD Threadripper PRO 5995WX processors.

GPU evaluation uses NVIDIA RTX A6000 and H100 devices, with additional CPU/Metal measurements in the supplement. The GPU gradient path uses FP32 working arrays in the reported comparisons, whereas independent score calculations use FP64. Wrong-split counts measure inferred nontrivial splits absent from the reference tree. For binary trees on the same leaves, normalized RF distance is this count divided by $N - 3$.

Table 1: Agreement with independent BME calculations. Errors are maximum absolute discrepancies over the stated tests.

Calculation	Test scope	Maximum error
CPU coordinate differences	36,000 changes, $N = 20$	3.553×10^{-14}
Ordered-tree enumeration	28,488 changes, $N = 4$–7	2.776×10^{-15}
Relabeling, first orientation	200 trees, $N = 25$	1.776×10^{-14}
Relabeling, reversed orientation	Same 200 trees	2.665×10^{-14}
Condensed SPR differences	16,231 changes, $N = 40$, $K = 16$	1.564×10^{-13}
FP32 CUDA gradient	Reported sizes $N = 100$–4,000	1.11×10^{-6}

Table 2: Single-core optimization at $N = 200$. Times are mean ± standard deviation in milliseconds over five replicates.

Processor	Distances	Reference	Phylo2Vec	Ratio
Apple M5	Additive	75.46 ± 2.42	18.14 ± 0.74	4.16
Apple M5	Noisy	85.16 ± 2.06	24.67 ± 2.79	3.45
AMD 5995WX	Additive	160.64 ± 5.51	27.96 ± 1.18	5.74
AMD 5995WX	Noisy	181.97 ± 4.30	36.91 ± 3.94	4.93

4 Results

4.1 Agreement with independent objective calculations

Table 1 summarizes the reported numerical checks. The largest discrepancy in the CPU coordinate tests is 3.553×10^{-14}, and the condensation tests agree to 1.564×10^{-13}. These are observed absolute errors on the tested cases, not universal error bounds. Rooted-vector round trips recover all 1,000 tested vectors at $N = 40$. Unrooted round trips preserve the split sets in a further 1,000 cases. Exhaustive rooted-tree enumeration at $N = 4, \ldots, 7$ checks 11,460 encodings.

Independent enumeration also identifies the scope of the coordinate neighborhood. On 30 random trees at each tested size from $N = 8$ to 20, a single ordered orientation covers approximately 27–33% of distinct SPR neighbors. Two orientations together cover approximately 52–56%. Neither constitutes an exhaustive SPR search. The full CPU optimizer's additional SPR scan is therefore material: independent enumeration found no improving SPR move in 90 returned trees at $N = 10, 16, 24$. This observation concerns that tested configuration, not the minibatch variant.

4.2 CPU optimization

Both methods recover the reference topology in all 60 noiseless trials. On the 60 noisy trials, Phylo2Vec obtains two lower BME scores, 57 ties, and one higher score than the in-memory reference. Thus it matches or improves the reference objective in 119 of 120 cases overall. A better BME score need not imply a smaller RF distance when the input distances are noisy. For example, the $N = 200$ noisy comparison has a lower mean reference RF error despite one BME-score win for Phylo2Vec.

At $N = 200$, the noiseless Apple M5 comparison requires 63.6 gradient evaluations on average, versus 211.0 reference SPR iterations. The corresponding mean times are 18.14 and 75.46 ms. Table 2 gives the largest-size timings for both processors. Supplementary Tables 4 and 5 give results at all sizes. External FastME comparisons at the same five sizes produce identical final unrooted splits and BME differences below 10^{-8} in the reported checks. These checks support the reference implementation on those inputs without making the in-memory timings measurements of the external executable.

4.3 Gradient throughput and memory

At $N = 4{,}000$, a reported full-gradient evaluation takes 6.219 ms on one H100, compared with 701.375 ms for its CPU reference. At $N = 40{,}000$, distributing the full-table computation across eight H100s reduces time from 816.34 to 158.36 ms, a factor of 5.15. The SPR stage alone falls from 756.26 to 98.28 ms. Column tiling reduces storage at the cost of recomputation: the reported one-GPU memory requirement falls from 31,142 to 13,614 MB, while the eight-GPU tiled evaluation takes 1,032.83 ms at this size. Thus the reduced-memory variant is useful when capacity, rather than latency, is limiting.

Distributed and streamed configurations extend a single full-tree sweep to $N = 380{,}000$, taking 205.450 s with 72,946 MB reported per GPU on eight H100s. This is a sweep benchmark, not the time to infer a converged tree. For the embedding-based workload with $N = 10{,}000{,}000$ and $K = 20{,}000$, reported average condensation and gradient times are 10.20 and 48.57 ms per batch on eight H100s. Multiplying their sum by 667 batches gives the reported epoch estimate of 39.20 s. It excludes any additional search work not represented by those two timings and does not establish a sequence-refinement time at that scale. The complete scaling tables are in Appendix G.

5 Discussion

The component identity provides the mathematical basis for evaluating local skeleton changes against a global BME objective. The unchanged internal terms form a constant, and the remaining terms are evaluated using exact balanced distances between components. This principle is established BME structure. Its role here is to make explicit which data a minibatch evaluator must preserve and to connect reduced skeleton optimization to Phylo2Vec coordinates and parallel execution.

The distinction between objective evaluation and search is essential. The distance matrix may itself be estimated, and the search may select only a small subset of candidates. Within the exact formulation, every evaluated coordinate difference is nevertheless the full-tree BME difference for that candidate. Conversely, a correct algebraic identity does not remove floating-point error or establish global convergence. Changing leaf labels enlarges the collection of coordinate neighborhoods available to the optimizer, but the tested two-orientation schedule alone is not an exhaustive SPR search.

The main computational tradeoff is between reuse and reconstruction. Dense tables permit rapid sweeps but require quadratic memory. Column tiling trades memory for repeated arithmetic. Skeletons reduce the dimension of the optimization problem, but exact construction of their distances can retain a dependence on the full leaf set. This becomes particularly visible in nonlinear sequence-distance refinement and in candidates involving large condensed components. Bilinear embedding benchmarks therefore establish a different scaling regime from the sequence experiments.

The empirical results support faster evaluation and competitive CPU optimization on the tested instances. They do not establish an asymptotic bound on the number of optimization sweeps, statistical superiority across data-generating models, or a universal numerical error bound. The supplementary sequence comparisons use approximate candidate distances followed by full-tree acceptance calculations and have no reported replicate uncertainty at the largest sizes. An end-to-end sequence benchmark using exact condensed distances for every candidate evaluation remains to be measured. Further comparisons should use matched distance matrices, identical starting trees, explicit timing boundaries, and repeated runs, with the exact implementation revision attached to each result.

6 Conclusion

Phylo2Vec coordinates provide a structured interface to exact, quadratic-time BME move evaluation. Exact balanced distances extend the same evaluator to reduced skeletons, preserving the full-tree change for every valid skeleton coordinate update. With these exact distances, minibatching restricts the set of candidate rearrangements while retaining the BME objective for each evaluated candidate. The computational cost depends on both the skeleton size and the construction of exact component distances. These identities support parallel tree search with exact objective differences, subject to numerical rounding, without implying a globally optimal search result.

References

[1] Y. Pauplin. Direct calculation of a tree length using a distance matrix. *Journal of Molecular Evolution*, 51(1):41–47, 2000.

[2] R. Desper and O. Gascuel. Fast and accurate phylogeny reconstruction algorithms based on the minimum-evolution principle. *Journal of Computational Biology*, 9(5):687–705, 2002.

[3] R. Desper and O. Gascuel. Theoretical foundation of the balanced minimum evolution method of phylogenetic inference and its relationship to weighted least-squares tree fitting. *Molecular Biology and Evolution*, 21(3):587–598, 2004. doi:10.1093/molbev/msh049.

[4] W. Hordijk and O. Gascuel. Improving the efficiency of SPR moves in phylogenetic tree search methods based on maximum likelihood. *Bioinformatics*, 21(24):4338–4347, 2005. doi:10.1093/bioinformatics/bti713.

[5] V. Lefort, R. Desper, and O. Gascuel. FastME 2.0: A comprehensive, accurate, and fast distance-based phylogeny inference program. *Molecular Biology and Evolution*, 32(10):2798–2800, 2015.

[6] M. J. Penn, N. Scheidwasser, M. P. Khurana, D. A. Duchêne, C. A. Donnelly, and S. Bhatt. Phylo2Vec: a vector representation for binary trees. *Systematic Biology*, 74(2):250–266, 2025. Advance access 2024. doi:10.1093/sysbio/syae030.

[7] D. C. Haws, T. L. Hodge, and R. Yoshida. Optimality of the neighbor joining algorithm and faces of the balanced minimum evolution polytope. *Bulletin of Mathematical Biology*, 73(11):2627–2648, 2011. doi:10.1007/s11538-011-9640-x.

[8] C. Piñeiro, J. M. Abuín, and J. C. Pichel. VeryFastTree: speeding up the estimation of phylogenies for large alignments through parallelization and vectorization strategies. *Bioinformatics*, 36(17):4658–4659, 2020.

[9] S. Mirarab, N. Nguyen, S. Guo, L.-S. Wang, J. Kim, and T. Warnow. PASTA: Ultra-large multiple sequence alignment for nucleotide and amino-acid sequences. *Journal of Computational Biology*, 22(5):377–386, 2015.

[10] M. J. Penn, N. Scheidwasser, C. A. Donnelly, D. A. Duchêne, and S. Bhatt. Bayesian inference of phylogenetic distances: revisiting the eigenvalue approach. *Bulletin of Mathematical Biology*, 87:32, 2025.

A Ordered representation and coordinate mapping

A.1 Linear-time decoding

For $v_i \le i$, initialize a component-root array by $r[x] = x$. Process $i = N-2, \dots, 0$. At step i, join the components with roots $r[v_i]$ and $r[i+1]$ at a new node $m = 2N-2-i$, then replace $r[v_i]$ by m. Before this step, the active components have distinct minimum labels $0, \dots, i+1$. Joining the two specified components removes minimum label $i+1$ and preserves minimum label v_i. Induction therefore establishes the invariant and the correctness of the construction. Each step uses constant work, giving an $O(N)$ decoder. This argument depends on the ordered condition and does not apply unchanged to a general vector with $v_i > i$.

A.2 Relabeling a decoded tree

Proposition A.1 (Ordered representation after relabeling)**.** *Given a decoded rooted binary tree, a parent-before-child ordering of its internal nodes constructs an ordered vector and a leaf permutation in $O(N)$ time. The represented labeled tree and its BME score are unchanged.*

Proof. Give the root temporary label zero and initialize a counter $t = 1$. At each internal node, choose an ordering (a, b) of its children. Pass the node's label to a, give b label t, record v_{t-1} as the inherited label, and increment t. Each newly issued label follows a unique chain of inheriting children to a leaf. Every leaf receives exactly one label, so the resulting labels form a permutation of $0, \dots, N-1$.

The inherited label was issued at an ancestor and is smaller than t, hence $v_{t-1} \le t-1$. To verify decoding, process the recorded splits in reverse order. All descendants of a node were assigned their new labels later than that node, so both child components have already been assembled when its recorded merge is reached. The merge therefore recreates that node's original clade. Induction reconstructs the entire tree. Permuting both axes of D by the inverse leaf-label map preserves all terms in Equation (1). Each vertex is processed a constant number of times. □

The implementation chooses the child order from the minimum original leaf label in each child component, with a second orientation reversing that choice. These are two deterministic coordinate systems for the same current tree. Their existence proves representational coverage after suitable relabeling, not coverage by a particular optimizer trajectory. For general candidate vectors, the implementation also uses a tree-to-vector encoder. Its unrooted version first inserts a degree-two root on the edge incident to leaf zero, so round-trip correctness is assessed by unrooted splits rather than root placement.

A.3 Mapping coordinate values to regraft edges

The ordered decoder numbers leaves $0, \dots, N-1$ and internal nodes $N, \dots, 2N-2$. The last index is the artificial root. Suppressing it leaves a computational root $R = 2N-3$ and the top edge joining R to the artificial root's other child, denoted c_{other}. Index ordinary edges by their child endpoints.

For row i, let $E_i[u]$ be the score change from pruning the component associated with leaf $i+1$ and regrafting it on the edge indexed by u. The half-tree SPR traversal supplies these values. Let $b_i(j)$ be the root of the component containing leaf j after the merges with coordinate index greater than i. Then the edge-to-coordinate map is

$$\Delta_{i,j} = \begin{cases} E_i[b_i(j)], & 0 \le j \le i, \\ E_i[(2N-2) - 2i + j], & i+1 \le j \le 2i-2, \\ E_i[c_{\text{other}}], & j \in \{2i-1, 2i\} \cap [i+1, 2i]. \end{cases} \tag{13}$$

The first case identifies the component on the corresponding pendant edge of the prefix tree. In the ordered decoder's node numbering, the remaining non-root prefix edges occupy consecutive indices $2N-1-i,\dots,2N-4$, giving the affine map in the second case. The last two rooted edge choices collapse onto the same unrooted top edge. At $i=1$, the middle case is empty and $j=1$ belongs to the first case. Only $j=2$ takes the collapsed-root value. Coordinate zero has no nontrivial alternative.

On a CPU, the array b_i can be maintained incrementally as the merge order advances. On a GPU, independent rows reconstruct their mappings using the latest eligible merge for each minimum leaf. When several merges have the same minimum leaf, only the latest eligible merge may write that destination. This avoids concurrent writes that would otherwise depend on thread scheduling.

B Parallel dependencies and workspace

B.1 Column ownership and ancestor traversal

For a fixed column w, the bottom-up update in Equation (4) reads only earlier entries of that column. Thus the recurrence has no cross-column dependency once its initial entries are available. The implementation separates the necessary transpose from the recurrence rather than writing both triangular copies concurrently. Top-down correction visits ancestors of w in order from the computational root. If a node's children have contiguous depth-first intervals, comparing the entry time of w with their interval boundary selects the child containing w. Only the current node and the fixed target are required as traversal state. The initial degree-three root update uses the other two root branches.

B.2 Autonomous row reconstruction

Store the balanced distances from internal vertices to leaves. For a requested row w, these entries supply its leaf values. For leaf w, the leaf values instead come directly from D. Average child entries bottom-up to fill internal positions, then overwrite ancestors of w top-down using the sibling and parent directions. Initialize the degree-three root correction explicitly and set the diagonal to zero.

For an internal position u in a disjoint component, the bottom-up average is the required balanced distance by linearity. For a descendant u of w, the stored leaf values already use the component at w directed away from that descendant, so averaging within the descendant gives the correct value. Ancestors of w require the complementary direction and are supplied by the top-down overwrite. These cases exhaust the row. The reconstruction costs $O(N+\mathrm{depth}(w))=O(N)$. Reconstructing a constant number of rows per pruned component therefore retains quadratic sweep complexity. The approximately fourfold reduction concerns the balanced-distance table only.

B.3 Traversal stack size

When a traversal reaches two outward components, push the larger component first and the smaller second on a last-in-first-out stack. The smaller component is processed next. Each extra pending larger component is associated with descent into a component at most half the size of its predecessor. Hence the number of simultaneously pending components is logarithmic in the number of tree vertices. Including the active traversal state gives a safe bound $\lfloor \log_2(2N-2) \rfloor + 2$. This is a workspace bound, not a statement that the traversal visits fewer vertices.

Table 3: RNASim refinement from a shared starting topology. VFT denotes 64-thread VeryFastTree minimum-evolution refinement. P2V denotes the sequence variant on one RTX A6000, using capped proposal distances above 2,048 leaves and full-tree acceptance checks. These are not timings of the exact-distance minibatch formulation. Times exclude the shared initialization.

N	Initial	Wrong splits		Refinement (s)		Init. (s)
	wrong splits	VFT	P2V	VFT	P2V	
1,000	214	178	176	6.88	0.06	0.58
5,000	890	703	696	33.38	0.68	3.40
10,000	1,667	1,313	1,320	65.98	1.55	7.82
50,000	6,729	4,980	4,950	313.53	16.84	63.01
100,000	12,375	8,857	8,802	593.50	104.45	179.03

C Sequence-refinement variant and recorded benchmarks

C.1 Implementation used for the recorded measurements

The sequence implementation used for the measurements in this appendix differs from the exact-distance search described in Section 2.5. For trees larger than 2,048 leaves, it caps the weighted leaf lists used for proposal distances at 64 entries per component. These lists are called profiles in the implementation. Above the cap, unresolved subtree mass is assigned to representative leaves, producing an approximate matrix $\widetilde{D}^{\mathrm{cap}}$. For smaller trees, the proposal calculation is uncapped.

The approximate matrix guides the inner skeleton search. Before accepting an update, the implementation recomputes affected condensed distances involving truncated components using every leaf contribution. Distances for untruncated pairs can be reused. It then evaluates Equation (11), with FP64 accumulation and a numerical acceptance margin of 10^{-6}. Outside weights are rebuilt from the current tree before each window, and ancestor summaries are updated after a commit. The acceptance calculation therefore evaluates the full-tree BME change, subject to rounding. It does not make the earlier proposal gradients exact, and the numerical margin is not a proved floating-point error bound.

These measurements characterize that implementation variant. They are not timings for constructing exact condensed distances and using them throughout every minibatch search.

C.2 Sequence data and comparison protocol

The recorded comparisons use the RNASim benchmark associated with PASTA [9], with simulated sequences and a known reference tree. The reported alignments have 1,596 columns. The minibatch comparison uses gap-aware JC69 distances. Selected full-tree experiments also use the eigenvalue-based distance approach [10], which changes the input objective and is identified separately. The CPU tools are 64-thread FastME 2.1.6 and VeryFastTree 4.0.5 [5, 8]. VeryFastTree's minimum-evolution refinement is run without its likelihood-refinement phase. Refinement time is reported separately from starting-tree construction. The minibatch times include the final full-tree acceptance calculation and sequential maintenance of the outside distances.

C.3 Recorded refinement results

Table 3 compares this minibatch variant with VeryFastTree refinement from the same initial topology. At $N = 50{,}000$, the minibatch method returns 4,950 incorrect splits in 16.84 s, compared with 4,980 in 313.53 s. At $N = 100{,}000$, the corresponding results are 8,802 in 104.45 s and 8,857 in 593.50 s. The refinement-time ratios are 18.6 and 5.7 respectively. These exclude the shared initialization, which takes 63.01 and 179.03 s. Adding that initialization to both methods gives approximately 4.72-fold and 2.73-fold ratios, rather than the refinement-only ratios.

The topological advantage is not uniform. At $N = 10{,}000$, the minibatch result has 1,320 incorrect splits, compared with 1,313 for VeryFastTree. A full-tree run with eigenvalue-based distances reaches 1,309 in 4.57 s, but uses a different distance estimate. The full-tree JC69 run returns 1,320 in 3.71 s. These variants should therefore not be combined into a single claim of consistently better topological accuracy.

At $N = 1{,}000$, FastME applied to the GPU-computed JC69 distance matrix returns 177 incorrect splits in 21.68 s with its own BIONJ initialization. Its shared-start refinement returns 178 in 11.27 s. The corresponding full-tree GPU result is 176 in 0.01 s, and minibatch refinement returns 176 in 0.06 s. The experiments differ in initialization and timing scope, so the ratios between their raw elapsed times are not interchangeable.

D Truncation and stale outside-distance counterexamples

Two constructions illustrate the need for exact condensed distances and current outside information. The first concerns the approximate-proposal variant in Appendix C. On a 4,102-leaf tree containing a balanced 4,096-leaf component, an adversarial split distance can give a capped proposal change of -0.25 while its true full-profile change is $+0.2421875$ for a cap of 64, or $+0.234375$ for a cap of 128. The proposal sign alone is therefore insufficient. Full-profile acceptance rejects both changes.

Separately, consider two disjoint windows whose internal rearrangements change their root-to-leaf weight vectors from p_A, p_B to $p_A + \delta p_A, p_B + \delta p_B$. With a fixed path coefficient κ between their attachment points, their cross contribution contains $\kappa p_A^\top D_{AB} p_B$. The jointly changed contribution includes the additional term

$$\kappa\, \delta p_A^\top D_{AB} \delta p_B,$$

which is absent from the sum of two changes computed against the original surroundings. In a 1,540-leaf test with two 768-leaf windows, changes -0.0032945313 and -0.0254489835 computed against frozen surroundings combine to an actual increase of $+0.0505258348$. Rebuilding the outside profile after the first commit makes the other proposal positive: $+0.0538203660$ or $+0.0759748183$, depending on the processing order. It is then rejected. These values come from controlled CPU replay of the acceptance and window-controller paths. They do not constitute a native-GPU concurrency test.

E Implementation and reproducibility

The CPU evaluator, tree encoders, and full-tree optimizer are implemented in `phylo2vec_fastme.c`, with input validation and Python bindings in `discrete_gradient.py`. The lower-triangular and full-coordinate interfaces require an ordered baseline. The multi-GPU evaluator and its tiled variants are implemented in `phylo2vec_cuda_8xh100.cu`. Sequence refinement is exercised by `bench_rnasim_gpu.cu`. The distinct distance-oracle recovery workload is implemented in `bench_minibatch_recovery.cu`.

Independent checks include `verify_all.c` for objective identities and encoding boundaries, `verify_spr_coverage.py` for neighborhood enumeration, and `compare_with_fastme_cli.py` for the external reference. The profile-completeness and window-composition constructions are recorded with their reproduction commands in the discrete-gradient review artifacts. The fifth review checks the window controller, full-profile arithmetic, and a shared-input batching path using CPU emulation. That harness does not cover the subsequent split-launch implementation. Native CUDA tests and the implementation revision associated with each timing must be recorded separately.

The present manuscript reorganizes existing measurements rather than reporting a new benchmark campaign. The source archive and revision notes accompanying this draft preserve the original tables and identify the sequence timings retained in Appendix C. An end-to-end sequence benchmark with exact condensed distances throughout is not supplied by those records.

Before submission, benchmark records should be consolidated with source revisions, compiler flags, seeds, repeat counts, and the exact timing boundaries. The earlier recovery and controlled-JC69 minibatch timings are retained in that archive rather than used to characterize the corrected acceptance path here.

F Supplementary CPU comparisons

Tables 4 and 5 give the full Apple M5 comparisons. FME denotes the in-memory reference implementation. P2V denotes Phylo2Vec. RF denotes normalized Robinson–Foulds distance. Wins and ties refer to the BME objective, with tolerance 10^{-9}. Timings and evaluation counts are means, with standard deviations and the reported P2V median and interquartile range retained.

Table 4: Additive distance comparison on Apple M5.

Objective and topology

N	FME L	P2V L	Win/tie/loss	FME RF	P2V RF
20	11.12739	11.12739	**0 / 20 / 0**	0.0000	0.0000
50	29.42572	29.42572	**0 / 20 / 0**	0.0000	0.0000
100	59.52693	59.52693	**0 / 10 / 0**	0.0000	0.0000
150	89.04365	89.04365	**0 / 5 / 0**	0.0000	0.0000
200	116.32398	116.32398	**0 / 5 / 0**	0.0000	0.0000

Computational cost

N	FME iters.	P2V evals.	FME ms	P2V ms	P2V med. [IQR]	Ratio
20	13.4	11.2	0.12 ± 0.02	0.08 ± 0.01	0.07 [0.02]	**1.54**×
50	42.2	19.4	1.58 ± 0.35	0.57 ± 0.13	0.59 [0.16]	**2.78**×
100	97.7	35.4	9.05 ± 0.84	2.63 ± 0.26	2.65 [0.44]	**3.44**×
150	156.2	49.4	31.47 ± 0.77	7.92 ± 0.86	7.75 [0.64]	**3.97**×
200	211.0	63.6	75.46 ± 2.42	18.14 ± 0.74	18.05 [1.17]	**4.16**×

Table 5: Noisy distance comparison on Apple M5.

Objective and topology

N	FME L	P2V L	Win/tie/loss	FME RF	P2V RF
20	11.19777	11.19777	**0 / 20 / 0**	0.0500	0.0500
50	29.25507	29.25607	**1 / 18 / 1**	0.0660	0.0649
100	59.25471	59.25471	**0 / 10 / 0**	0.0526	0.0526
150	92.14846	92.14846	**0 / 5 / 0**	0.0395	0.0395
200	120.94220	120.94220	**1 / 4 / 0**	0.0467	0.0487

Computational cost

N	FME iters.	P2V evals.	FME ms	P2V ms	P2V med. [IQR]	Ratio
20	14.4	12.4	0.07 ± 0.01	0.05 ± 0.01	0.04 [0.02]	**1.60**×
50	46.1	25.9	1.07 ± 0.07	0.49 ± 0.09	0.49 [0.10]	**2.18**×
100	102.6	41.0	10.62 ± 2.53	3.32 ± 0.56	3.18 [0.52]	**3.19**×
150	166.0	61.0	33.97 ± 1.01	10.15 ± 1.35	10.17 [1.35]	**3.35**×
200	235.8	84.4	85.16 ± 2.06	24.67 ± 2.79	24.51 [4.48]	**3.45**×

G Supplementary scaling measurements

The tables below retain the reported sweep measurements, separating CPU/Metal, CUDA, sharded, and embedding-based workloads. The single-GPU speedups use their respective host CPU references. They should not be interpreted as comparisons against one common host baseline. Stage timings measure balanced-distance construction and SPR evaluation, rather than optimization to convergence. The $2N^2$ column in the sharded table is the original rectangular-workload size proxy. It is not the number of valid Phylo2Vec coordinate entries, which is $(N-1)^2$, nor a count of distinct neighboring trees. Reported memory units are retained from the benchmark records.

For the embedding workload, each batch has 20,000 skeleton leaves, comprising 15,000 expanded leaves and 5,000 condensed components. Its epoch columns multiply average condensation-plus-gradient time by $\lceil N/15{,}000 \rceil$. They are workload estimates rather than measured convergence times. The largest listed validation discrepancy in this workload is 1.4×10^{-5}, and the listed validation outcomes refer to the benchmark's own checks.

Table 6: CPU and Metal gradient timings on Apple M5. All times are milliseconds.

FP64 full sweep

N	1 CPU	10 CPU	Ratio	10 CPU tiled
100	0.055	0.068	0.80×	0.096
250	0.332	0.322	1.03×	0.246
500	1.273	0.433	**2.94**×	0.508
1,000	5.860	1.878	**3.12**×	2.081
2,000	27.399	10.230	**2.68**×	10.806
3,000	85.815	24.428	**3.51**×	**23.777 ms**
4,000	132.225	51.265	**2.58**×	**43.329 ms**

Balanced-distance construction

N	CPU FP64	CPU FP32	Metal FP32	Ratio
100	0.012	0.011	0.517	0.02×
250	0.076	0.058	0.906	0.08×
500	0.312	0.223	1.216	0.26×
1,000	1.565	1.344	2.699	0.58×
2,000	8.954	9.533	4.462	**2.01**×
3,000	22.504	35.030	8.970	**2.51**×
4,000	68.188	59.289	7.571	**9.01**×

Hybrid sweep and absolute errors

N	Hybrid FP32	Ratio	FP64 error	FP32 error
100	0.524	0.10×	0.00e+00	4.35e-07
250	0.934	0.36×	0.00e+00	5.24e-07
500	1.742	0.73×	5.55e-17	6.63e-07
1,000	3.097	**1.89**×	4.44e-16	8.11e-07
2,000	6.312	**4.34**×	4.44e-16	8.93e-07
3,000	14.359	**5.98**×	5.27e-16	9.84e-07
4,000	26.922	**4.91**×	5.00e-16	1.13e-06

Table 7: Single-GPU gradient timings and agreement with the CPU reference. Times are milliseconds.

RTX A6000 timings and reported maximum errors

N	A6000 full	Ratio	A6000 tiled	Max. error
100	0.061	**8.44**×	0.078	3.68e-07
250	0.186	**15.92**×	0.207	6.22e-07
500	0.382	**32.95**×	0.511	6.48e-07
1,000	1.035	**33.69**×	1.988	9.29e-07
2,000	3.851	**40.15**×	7.288	1.11e-06
4,000	11.053	**62.48**×	43.124	1.07e-06

H100 timings

N	CPU ref.	H100 avg.	H100 SPR	H100 total	Ratio	H100 tiled
100	0.249	0.044	0.027	0.071	**3.51**×	0.095
250	1.964	0.116	0.048	0.164	**11.98**×	0.205
500	8.176	0.227	0.077	0.304	**26.89**×	0.460
1,000	32.336	0.661	0.105	0.766	**42.21**×	1.030
2,000	154.091	1.557	0.726	2.283	**67.50**×	4.273
4,000	701.375	3.565	2.654	6.219	**112.78**×	22.327

The maximum-error column is the reported combined check across the GPU paths. All listed checks pass their benchmark tolerance. CPU baselines differ between hosts.

Table 8: Full-table and tiled multi-GPU sweeps. Times are milliseconds and memory is reported in MB.

A6000 timings and memory columns

N	2 A6000 full	2 A6000 tiled	Full memory	Tiled memory
1,000	0.93	5.75	27	39
2,000	4.89	13.90	92	93
4,000	14.69	56.85	336	247
10,000	90.45	249.59	1,985	1,076
20,000	422.73	1,212.27	7,785	3,677
40,000	*OOM*	5,628.62	31,142	13,614

Full-table H100 sweep

N	1 H100 [avg.+SPR]	8 H100	Ratio
1,000	0.76 [0.65 + 0.11]	0.79	0.96×
2,000	2.21 [1.47 + 0.74]	1.71	**1.29**×
4,000	6.01 [3.57 + 2.44]	3.98	**1.51**×
10,000	29.61 [10.10 + 19.50]	13.03	**2.27**×
20,000	155.93 [23.23 + 132.70]	41.06	**3.80**×
40,000	816.34 [60.08 + 756.26]	**158.36**	**5.15**×

Tiled H100 sweep

N	1 H100 tiled	8 H100 tiled	Top-1 check
1,000	1.46	4.47	**PASS** (0.00e+00)
2,000	6.14	5.21	**PASS** (2.38e-07)
4,000	30.78	10.98	**PASS** (0.00e+00)
10,000	250.69	57.53	**PASS** (0.00e+00)
20,000	1,123.43	244.13	**PASS** (4.77e-07)
40,000	4,768.31	**1,032.83**	**PASS** (0.00e+00)

Table 9: Distributed sweeps on eight H100 GPUs.

Timing

N	Work proxy $2N^2$	Distance ms	SPR ms	Total s
40,000	3.20×10^9	514.54	517.92	**1.032**
80,000	1.28×10^{10}	2,112.82	2,321.28	**4.434**
120,000	2.88×10^{10}	4,785.75	4,803.60	**9.589**
180,000	6.48×10^{10}	10,848.67	13,977.73	**24.826**
250,000	1.25×10^{11}	39,981.55	43,721.48	**83.703**
350,000	2.45×10^{11}	76,240.05	96,515.83	**172.756**
380,000	2.89×10^{11}	89,992.00	115,457.57	**205.450**

Memory and execution mode

N	MB/GPU	Aggregate GB	Mode and check
40,000	2,386	18.6	`Full-Shard` [**PASS**]
80,000	7,824	61.1	`Shard+Calib` [**PASS**]
120,000	16,314	127.5	`Shard+Calib` [**PASS**]
180,000	34,770	271.6	`Shard+Calib` [**PASS**]
250,000	62,296	486.7	`Shard+Calib` [**PASS**]
350,000	62,180	485.8	`StreamD+Calib` [**PASS**]
380,000	72,946	569.9	`StreamD+Calib` [**PASS**]

At $N = 400{,}000$, the reported requirement is 80,600 MB/GPU (629.7 GB aggregate), exceeding available device memory. Full-Shard and Shard+Calib retain distance shards. StreamD+Calib generates distance tiles as required. The work proxy is not a count of distinct moves.

Table 10: Embedding-based condensed sweeps with $K = 20{,}000$ skeleton leaves. The epoch columns are batch-count times mean condensation-plus-gradient time.

Two A6000 GPUs

N	Batches	2 A6000 cond.+grad. (ms)	Epoch (s)
1,000,000	67	$4.45 + 284.44$	19.36
5,000,000	334	$8.42 + 283.02$	97.34
10,000,000	667	$17.29 + 281.58$	199.35

Eight H100 GPUs

N	Cond. ms	Grad. ms	Epoch s	MB/GPU	Check
1,000,000	2.69	48.53	**3.43**	7,793	**PASS** (1.4e-05)
5,000,000	4.86	48.48	**17.82**	7,824	**PASS** (2.1e-06)
10,000,000	10.20	48.57	**39.20**	7,862	**PASS** (2.2e-06)

This workload uses a bilinear embedding distance and is separate from sequence-distance refinement. The validation figures are observed discrepancies, not universal error bounds.